CLAUDE CERTIFIED ARCHITECT — FOUNDATIONS

Objective 36: MCP tool descriptions

> **Exam objective:** Write MCP tool descriptions that clearly distinguish each tool's purpose, input formats, use-case boundaries, and relationships to semantically similar tools, reducing misrouting and incorrect tool selection.

Q1.

An MCP server exposes two tools: `search_customer` and `get_customer_by_id`. Models frequently call `search_customer` even when an exact numeric ID is present. Which description improvement is most likely to reduce this misrouting?

A. Make both descriptions shorter and nearly identical B. State that `get_customer_by_id` should be used when a stable customer ID is known, while `search_customer` is for ambiguous name, email, or text lookups C. Remove the ID format from the description to keep it generic D. Merge both tools into one description but keep separate schemas

Answer: B **Explanation:** The model needs a clear decision boundary between exact retrieval and exploratory search. Explicitly describing when an ID is available versus when fuzzy lookup is needed reduces overlap. Making descriptions generic or identical increases ambiguity rather than resolving it.

Q2.

A tool named `create_invoice` expects a JSON object with `customerId`, `lineItems`, and `currency`, but users often phrase requests in natural language. Which description pattern best helps the model supply the right inputs?

A. "Creates invoices as needed." B. "Use to generate billing artifacts." C. "Creates a new invoice. Requires structured fields: `customerId` (string), `lineItems` (array of item objects), and `currency` (ISO code). Do not use for quote generation or invoice lookup." D. "Invoice tool. Flexible input accepted."

Answer: C **Explanation:** A strong tool description names the purpose, required input shape, and non-goals. This helps the model translate user intent into the expected schema and avoid using the tool for related but different tasks like quotes or lookups. Vague descriptions make misfires more likely.

Q3.

Two tools exist: `list_open_tickets` and `search_tickets`. The first returns current unresolved tickets for a specific customer account; the second does full-text search across all tickets. Which

description strategy most clearly separates them?

A. Describe both as "find tickets" B. Emphasize that `list_open_tickets` is account-scoped and status-filtered, while `search_tickets` is repository-wide full-text discovery for uncertain cases C. Remove the scope details because the schema already contains fields D. Add implementation details about database indexes only

Answer: B **Explanation:** Tool descriptions should explain intent and boundaries, not just internal mechanics. Account-scoped open-ticket listing is a different task from broad investigative search, and that distinction belongs in the description. Relying on schema fields alone often leaves the model without a clear selection rule.

Q4.

An engineer writes the following description: "Fetches records fast." What is the primary problem with this description?

A. It reveals too much about latency expectations B. It omits purpose, input semantics, and boundaries needed to distinguish the tool from others C. It is too long for models to parse D. It prevents authentication from working

Answer: B **Explanation:** "Fetches records fast" says almost nothing about what records, when to use the tool, or what inputs it expects. Effective descriptions reduce ambiguity by clarifying scope and contrast with adjacent tools. Performance claims alone do not guide selection.

Q5.

A server has tools `download_report_pdf` and `get_report_metadata`. Models sometimes call the PDF download tool when the user only asks for date, owner, and status. Which wording change is best?

A. Describe `download_report_pdf` as "Returns the full binary report document; use only when file content is needed, not for summary fields" B. Remove the word "report" from both tools C. Rename `get_report_metadata` to `tool2` D. Add the same examples to both descriptions

Answer: A **Explanation:** The description should clarify that one tool returns heavy document content while the other serves lightweight inspection needs. This boundary helps the model avoid unnecessary file retrieval. Removing names or duplicating examples increases confusion instead of reducing it.

Q6.

Which description best helps a model choose between `append_note_to_case` and `update_case_status`?

A. "`append_note_to_case`: modifies a case. `update_case_status`: modifies a case." B. "`append_note_to_case`: adds a textual note to the activity history without changing workflow state. `update_case_status`: changes the official workflow state such as Open, Pending, or Closed." C. "`append_note_to_case`: note tool. `update_case_status`: status tool." D. "Both tools update case data; use whichever seems appropriate."

Answer: B **Explanation:** Good descriptions identify the exact type of change each tool makes and the consequences of using it. Adding commentary versus changing workflow state are meaningfully different actions. The other choices collapse that distinction and invite incorrect tool selection.

Q7.

A tool expects ISO 8601 timestamps, but the description only says "pass the date." As a result, models send locale-specific strings like "03/04/26". What is the best fix?

A. Add explicit input format guidance such as "Use ISO 8601 timestamps like `2026-04-03T14:00:00Z`" B. Remove all date references from the description C. Increase the tool timeout D. Depend on the model to normalize all dates automatically

Answer: A **Explanation:** When input formats are strict, the description should name them clearly and preferably include an example. Ambiguous date wording causes parsing failures and locale confusion. Timeout changes and wishful normalization do not solve a schema mismatch.

Q8.

A platform team wants descriptions that reduce errors between `delete_user_session` and `revoke_user_access`. Which extra detail is most valuable?

A. The color theme of the admin console each tool uses B. A warning that `delete_user_session` ends current sessions, while `revoke_user_access` removes future authorization and is a broader account action C. The programming language used to implement each endpoint D. The number of unit tests behind each tool

Answer: B **Explanation:** Related tools often need explicit scope boundaries and consequences. Ending current sessions is a narrower operational action than revoking account access, and the description should say so. UI themes and implementation details do not guide correct routing.

Q9.

An architect is reviewing a tool description that includes many internal service names but never explains user-facing intent. What is the likely outcome?

A. Better tool selection because models prefer internal jargon B. Higher risk of misrouting because the description lacks clear task semantics C. Automatic schema validation D. Lower latency during tool execution

Answer: B **Explanation:** Models choose tools based on the semantic mapping from user request to tool purpose. Heavy internal jargon without user-facing intent obscures that mapping. Validation and latency are unrelated to description clarity.

Q10.

Why is it useful for a tool description to reference semantically similar tools, such as stating "Use `search_orders` for broad text search; use this tool only for exact order-number retrieval"?

A. It gives the model a comparative boundary that reduces overlap-based confusion B. It increases randomness so the model explores more tools C. It replaces the need for an input schema D. It guarantees the tool will always be invoked

Answer: A **Explanation:** Comparative guidance is one of the strongest ways to disambiguate neighboring tools. It tells the model not just what this tool does, but why it should choose it over a similar alternative. It does not replace schemas or force invocation by itself.

Q11.

An MCP team is tempted to put every operational detail, exception case, and backend dependency into each description. What is the best critique?

A. Descriptions should be detailed enough to guide selection, but overloaded descriptions can bury the key purpose and decision boundaries B. More text is always better because models never miss the main point C. Backend dependency names are the only important part D. Tool descriptions should contain authentication secrets for convenience

Answer: A **Explanation:** Effective descriptions are clear and specific, not bloated. They should prioritize purpose, required input shape, and boundaries with nearby tools. Excess implementation detail can dilute the guidance the model needs most.

Q12.

Which description most appropriately defines use-case boundaries for a tool named `generate_refund_quote`?

A. "Refund-related operations." B. "Calculates an estimated refund amount for review before approval. Does not issue the refund, change ledger balances, or contact payment providers." C. "Handles payments and customer changes." D. "General finance utility."

Answer: B **Explanation:** This description clearly states both what the tool does and what it explicitly does not do. That prevents the model from confusing estimation with execution. The other descriptions are too broad to support reliable routing.

Q13.

A support workflow exposes `get_incident_summary` and `get_incident_timeline`. Users ask, "What happened during the outage between 2:00 and 2:30?" Which description change most helps the model pick the timeline tool?

A. Note that `get_incident_timeline` returns ordered event entries over specified time ranges, while `get_incident_summary` gives a high-level overview B. Rename both tools to start with `incident_` C. Remove mention of time windows from the timeline tool D. Add the same phrase "incident details" to both tools

Answer: A **Explanation:** The user's request implies a time-bounded sequence of events, which should map to the timeline tool. The description needs to surface that distinction explicitly. Making names more similar or removing time-window guidance would worsen selection.

Q14.

An engineer claims input formats belong only in schemas, never in descriptions. Why is that incomplete?

A. Because descriptions help the model understand which schema fields matter and how values should be represented before it attempts a call B. Because schemas are ignored by clients C. Because descriptions automatically validate inputs D. Because descriptions can replace server-side validation

Answer: A **Explanation:** Schemas define structure, but descriptions help the model map language into that structure and choose among overlapping tools. Mentioning critical formats in plain language often reduces preventable mistakes. Validation is still required elsewhere.

Q15.

You are evaluating two candidate descriptions for a tool named `upload_knowledge_article`. Which is better?

A. "Uploads things to the knowledge system." B. "Publishes a new knowledge-base article from provided title, body, and category fields. Use for creating new articles, not for editing existing ones or uploading binary attachments." C. "Knowledge tool." D. "Sends content somewhere if needed."

Answer: B **Explanation:** The best description combines clear purpose, expected inputs, and sharp boundaries against adjacent tasks like editing or attachment upload. That is exactly the information models need for correct selection. The alternatives are too vague to prevent misrouting.
